

CampaignVoucher - Cost Estimate (SPEC Section 7/8) - - scenario: normal

As of block 1, timestamp 1.

Methodology

Gas figures below are measured directly, with no mocks: the full contract system (Groth16 verifier, EntryPoint v0.7, CitizenFactory, CitizenAccount, VoucherPaymaster, VoucherNFT) is deployed fresh and driven through SPEC Section 4's canonical walkthrough (create/claim/donate sponsored through the real EntryPoint + paymaster; spend/redeem as plain EOA calls), identical to `test/EndToEnd.t.sol`. This report covers exactly one price scenario -- run this script again with a different `L1_SCENARIO` (or with `--fork-url` for Base) to get the others; see the file list under gas-reports/. For an L1 scenario, `block.basefee` is pinned via `vm.fee(...)` to that scenario's own price BEFORE deployment, so the measured gas reflects that exact price -- not a value borrowed from whatever chain happened to be forked (EntryPoint's own fee accounting reads `block.basefee` directly, so gas usage is not perfectly price-independent). For Base, the real `GasPriceOracle` precompile and live `block.basefee` are used instead (`--fork-url <base-rpc>` required). ETH_USD is a supplied parameter (via `.env`, see `.env.example`), not measured or invented by the script.

Parameters

Parameter	Value	Source
Scenario	normal	interactive prompt
ETH_USD	\$1850	env var (default 1850, researched 2026-08-04)
L1 gas price	0.500 gwei	env var, pinned via <code>vm.fee(...)</code> before measurement (researched 2026-08-04)

Per-action cost (SPEC Section 4/7)

Action	Gas	Cost (normal)
Create	832608	\$0.770162
Claim	140167	\$0.129654
Donate	80193	\$0.074178
Spend (1)	40023	\$0.037021
Redeem (1)	10845	\$0.010031

(1) on Spend/Redeem means a single voucher, not a fixed per-call cost: both `batchTransfer/redeem` take an array of voucher IDs and can sweep many at once (SPEC's batching requirement at the candidate->vendor and vendor->authority outflows), so this is the no-batching baseline, not what a real accumulated sweep

costs -- see B3 ([test/gas/B3_OutflowSlope.t.sol](#)) for the measured marginal gas/voucher once batched, and Section 8 below for that fit applied to a real population.

([a_register](#) measured separately: 24066 gas -- used only in Section 8 below.)

Program-scale extrapolation (SPEC Section 8)

User-provided population: 5000000 citizens, 500 candidates, 10000 vendors, 100% of citizens donate their voucher, each candidate/vendor sweeps its accumulated vouchers in 1 / 1 batched transaction(s) respectively.

B3 refit: spend slope=12745 gas/voucher, intercept=27051; redeem slope=8745 gas/voucher, intercept=2111.

Sponsor-paid and self-paid are collapsed to two buckets, not three: candidate/vendor registration is authority-paid (SPEC Section 3.1's AllowList admin model), the same party that funds the paymaster deposit (SPEC Section 3.6) -- so it's counted as sponsor-paid, not a separate bucket.

Extrapolated total gas: 5372577328500

Bucket	Gas	Cost (normal)
Sponsor-paid (create+claim+donate+registration)	5265092693000	\$4870210.741025
Self-paid (candidate/vendor batched spend+redeem)	107484635500	\$99423.287837
Program total	5372577328500	\$4969634.028862